

AI Assisted Coding Ass-16.3

B.Sushanth

2403A52L11-B50

Task1:

Prompt:

Design a normalized SQL schema for a Hospital Management System with Doctors, Patients, and Appointments tables.

Include primary keys, foreign keys, and constraints.

Generate SQL queries using joins for reporting.

Code:

```
week16.3 > task1.sql
1  CREATE TABLE Doctors (
2      doctor_id INT PRIMARY KEY,
3      name VARCHAR(100),
4      specialization VARCHAR(100)
5  );
6
7  CREATE TABLE Patients (
8      patient_id INT PRIMARY KEY,
9      name VARCHAR(100),
10     age INT
11 );
12
13 CREATE TABLE Appointments (
14     appointment_id INT PRIMARY KEY,
15     doctor_id INT,
16     patient_id INT,
17     appointment_date DATE,
18     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
19     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
20     CHECK (appointment_date >= CURRENT_DATE)
21 );
```

Task2:

Prompt:

Design an e-commerce database schema with Users, Products, Orders, and OrderDetails.

Ensure normalization and generate analytical queries.

Code:

```
week16.3 > task2.sql
1  CREATE TABLE Users (
2      user_id INT PRIMARY KEY,
3      name VARCHAR(100),
4      email VARCHAR(100) UNIQUE
5  );
6
7  CREATE TABLE Products (
8      product_id INT PRIMARY KEY,
9      name VARCHAR(100),
10     price DECIMAL(10,2)
11 );
12
13 CREATE TABLE Orders (
14     order_id INT PRIMARY KEY,
15     user_id INT,
16     order_date DATE,
17     FOREIGN KEY (user_id) REFERENCES Users(user_id)
18 );
19
20 CREATE TABLE OrderDetails (
21     order_detail_id INT PRIMARY KEY,
22     order_id INT,
23     product_id INT,
24     quantity INT,
25     FOREIGN KEY (order_id) REFERENCES Orders(order_id),
26     FOREIGN KEY (product_id) REFERENCES Products(product_id)
27 );
28 -- 1. Orders by user
29 SELECT o.*
30 FROM Orders o
31 WHERE o.user_id = 1;
32 -- 2. Most purchased product
33 SELECT p.name, SUM(od.quantity) AS total_sold
34 FROM OrderDetails od
35 JOIN Products p ON od.product_id = p.product_id
36 GROUP BY p.name
37 ORDER BY total_sold DESC
38 LIMIT 1;
```

Task3:

Prompt:

Design a library database with Books, Members, and Loans.

Suggest indexing and write queries for issued and overdue books.

Code:

```
week16.3 > task3.sql
1  CREATE TABLE Books (
2      book_id INT PRIMARY KEY,
3      title VARCHAR(200),
4      author VARCHAR(100)
5  );
6
7  CREATE TABLE Members (
8      member_id INT PRIMARY KEY,
9      name VARCHAR(100)
10 );
11
12 CREATE TABLE Loans (
13     loan_id INT PRIMARY KEY,
14     book_id INT,
15     member_id INT,
16     loan_date DATE,
17     return_date DATE,
18     FOREIGN KEY (book_id) REFERENCES Books(book_id),
19     FOREIGN KEY (member_id) REFERENCES Members(member_id)
20 );
21 -- 1. Books currently on loan
22 SELECT b.title, m.name
23 FROM Loans l
24 JOIN Books b ON l.book_id = b.book_id
25 JOIN Members m ON l.member_id = m.member_id
26 WHERE l.return_date IS NULL;
27 -- 2. Overdue books
28 SELECT b.title, m.name, l.loan_date
29 FROM Loans l
30 JOIN Books b ON l.book_id = b.book_id
31 JOIN Members m ON l.member_id = m.member_id
32 WHERE l.return_date IS NULL
33 AND l.loan_date < CURRENT_DATE - INTERVAL 30 DAY;
```

Task 4 – Query Optimization

```

week16.3 > task4.sql
1  ♦ Given
2  SELECT * FROM Books
3  WHERE author_id IN (
4  |     SELECT author_id FROM Authors WHERE country='UK'
5  | );
6
7  ♦ Optimized
8  SELECT b.*
9  FROM Books b
10 JOIN Authors a ON b.author_id = a.author_id
11 WHERE a.country = 'UK';
12

```

Task 5 – University Course Registration

◆ Prompt

Design a normalized schema for Students, Courses, Faculty, and Registrations. Define relationships and write analytical queries.

◆ SQL

```

week16.3 > task5.sql
1  CREATE TABLE Faculty (
2  |     faculty_id INT PRIMARY KEY,
3  |     name VARCHAR(100)
4  | );
5
6  CREATE TABLE Students (
7  |     student_id INT PRIMARY KEY,
8  |     name VARCHAR(100)
9  | );
10 CREATE TABLE Courses (
11 |     course_id INT PRIMARY KEY,
12 |     course_name VARCHAR(100),
13 |     faculty_id INT,
14 |     FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
15 | );
16 CREATE TABLE Registrations (
17 |     reg_id INT PRIMARY KEY,
18 |     student_id INT,
19 |     course_id INT,
20 |     FOREIGN KEY (student_id) REFERENCES Students(student_id),
21 |     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
22 | );
23 -- 1. Students in a course
24 SELECT s.name
25 FROM Registrations r
26 JOIN Students s ON r.student_id = s.student_id
27 WHERE r.course_id = 101;
28 -- 2. Faculty teaching > 2 courses
29 SELECT f.name, COUNT(c.course_id) AS total_courses
30 FROM Faculty f
31 JOIN Courses c ON f.faculty_id = c.faculty_id
32 GROUP BY f.name
33 HAVING COUNT(c.course_id) > 2;
34 -- 3. Most popular courses
35 SELECT c.course_name, COUNT(r.student_id) AS total_students
36 FROM Courses c
37 JOIN Registrations r ON c.course_id = r.course_id
38 GROUP BY c.course_name
39 ORDER BY total_students DESC;

```